

```

1  #include <stdio.h>
2
3  #define max 100 // sos cung toi da
4
5  typedef int Element;
6
7  typedef struct {
8      Element data[max];
9      int size;
10 }List;
11
12 // cau truc cua do thi G
13 typedef struct{
14     int n; // so dinh vaf so cung cua do thi
15     List adj[max];
16 }Graph;
17
18
19 void make_null(List *L){
20     L->size = 0;
21 }
22
23 void push(List *L, Element x){
24     L->data[L->size] = x;
25     L->size++;
26 }
27
28 Element element_at(List * L, int x){
29     return L->data[x-1];
30 }
31
32 int countList(List *L){
33     return L->size;
34 }
35 // ham khoi tao do thi
36 void init_Graph(Graph *G, int n){
37     G->n = n; // gan so dinh cho do thi
38     int u;
39     for(u = 1; u <= n; u++){
40         make_null(&G->adj[u]);
41     }
42
43 // ham them cung cho do thi
44 void add_Edge(Graph *G, int x, int y){
45     push(&G->adj[x],y);
46     push(&G->adj[y],x);
47 }
48
49 int adjacent(Graph *pG, int u, int v) {
50     int j;
51     for (j = 1; j <= pG->adj[u].size; j++)
52         if (element_at(&pG->adj[u], j) == v)
53             return 1; //v n?m trong danh sách adj[u]
54     return 0; //v không có trong danh sách adj[u]
55 }
56
57 void neighbours(Graph *G, int u){
58     int v;
59     for(v = 1; v<= G->n; v++){
60         if(adjacent(G,u,v) != 0)
61             printf("%d ", v);
62         printf("\n");
63     }
64
65 int main(){
66
67     Graph G;
68     int x,y,e;
69     // FILE *f = fopen("dothi.txt", "r");

```

```

70 // fscanf(f, "%d%d", &x, &y);
71 // init_Graph(&G, x);
72 // int u, v;
73 // for(e = 1; e<= y; e++){
74 //     fscanf(f, "%d%d", &u,&v);
75 //     add_Edge(&G, u, v);
76 // }
77 int u, v;
78 freopen("dothi1.txt", "r", stdin);
79 scanf("%d%d", &x,&y);
80 init_Graph(&G, x);
81 for(e = 1; e<=y; e++)
82 {
83     scanf("%d%d", &u, &v);
84     add_Edge(&G, u,v);
85 }
86 // init_Graph(&G, 4);
87 // add_Edge(&G, 1,2);
88 // add_Edge(&G, 2,3);
89 // add_Edge(&G, 3,4);
90 // add_Edge(&G, 4,1);
91 // add_Edge(&G, 2,4);
92 int i,j;
93
94 printf("-----\n");
95 int a;
96 for(a = 1; a <= G.n; a++)
97 for(j = 1; j <=G.n; j++){
98     if(adjacent(&G, a, j) == 1){
99         printf("%d ke %d\n", a, j);
100     }
101     else {
102         printf("%d khong ke %d\n", a, j);
103     }
104 }
105
106 for(i = 1; i <= G.n; i++){
107     printf("Neighbours cua %d la: ", i);
108     neighbours(&G, i);
109     printf("\n");
110 }
111 return 0;
112
113
114 }
115

```